FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

Edge LLM Optimizaion

Sadra Farshid



Supervisor:

March, 2026

Table of Contents

Introduction.....3
Literature Review.....5
Proposed Methodology.....11
Implementation.....18
Evaluates.....32
Concludes.....33
References.....34
Attached.....36

Introduction

The rapid growth of the Internet of Things (IoT) has transformed modern living environments, particularly in the context of smart homes, where interconnected devices continuously collect and exchange data to enhance user comfort, energy efficiency, and automation. Smart home systems rely on a diverse range of sensors, actuators, and intelligent controllers to monitor and manage household activities, leading to the generation of large-scale, heterogeneous, and dynamic data streams.

In recent years, machine learning (ML) and deep learning approaches have been widely adopted to process IoT data for tasks such as anomaly detection, activity recognition, and energy optimization. However, traditional ML models often struggle with generalization, adaptability, and interpretability, particularly in dynamic environments such as smart homes. Moreover, centralized cloud-based solutions introduce significant challenges, including latency, privacy concerns, and high computational costs.

The emergence of Large Language Models (LLMs) has opened new opportunities for intelligent data processing and decision-making. LLMs demonstrate strong capabilities in reasoning, representation learning, and knowledge generalization across domains. Recent studies have explored the role of LLMs as meta-optimizers, enabling automated model tuning and adaptive learning strategies. Specifically, in smart home environments, LLM-based systems have been proposed to improve personalization, optimize energy consumption, and provide context-aware automation. These systems can transform raw sensor data into actionable insights and continuously refine performance through feedback mechanisms.

Despite these advancements, deploying LLMs in smart home environments presents significant challenges. Most existing approaches rely on cloud-based architectures, which are not suitable for latency-sensitive and privacy-critical applications. Furthermore, LLMs are computationally intensive, making deployment on resource-constrained devices difficult. Therefore, this research focuses on optimizing offline LLM models for smart home data. The proposed approach aims to improve efficiency, enable local inference, and enhance interpretability. This work contributes to developing a scalable and privacy-preserving intelligent smart home system.

between raw data collection and meaningful user interactions. By leveraging advanced language understanding and reasoning capabilities, LLMs can interpret complex sensor data patterns, generate natural language explanations, and provide predictive insights that support proactive decision-making. This capability not only enhances user experience but also contributes to developing systems that can adapt to changing environments and user behaviors, addressing one of the key limitations of traditional IoT data analytics.

Moreover, optimizing LLMs for offline and edge deployment ensures that smart home systems can operate with minimal reliance on cloud infrastructure. This approach reduces latency, preserves user privacy, and decreases dependency on continuous internet connectivity, making the system more robust and reliable. The proposed research explores novel techniques for model compression, knowledge distillation, and efficient inference, aiming to create lightweight yet powerful LLM-based models suitable for deployment on resource-constrained devices. These advancements are expected to enable next-generation smart home solutions that are both intelligent and practical for real-world applications.

The remainder of this thesis is organized as follows. Chapter 2 reviews related work. Chapter 3 presents the proposed methodology. Chapter 4 describes implementation details. Chapter 5 evaluates the results. Finally, Chapter 6 concludes the thesis.

Literature Review

The Internet of Things (IoT) has significantly transformed modern digital ecosystems by enabling seamless communication between interconnected devices. Smart home environments represent one of the most prominent applications of IoT, where sensors, actuators, and intelligent systems collaborate to automate household activities and improve user experience. However, these systems generate large-scale, heterogeneous, and dynamic data streams, which introduce challenges in data processing, system scalability, and real-time decision-making [1]. Recent studies have highlighted that smart home environments require adaptive and intelligent models capable of handling context-aware data and dynamic user behavior. Traditional rule-based systems and conventional machine learning models often fail to capture the complexity of such environments [2].

Francesco D'Amore [3] explores the use of quantum machine learning techniques, specifically Projected Quantum Kernels (PQK), for analyzing real-world IoT data. The main motivation of the study is to address a key limitation in quantum machine learning research, namely the lack of suitable datasets that can be directly processed by quantum models without heavy preprocessing. To overcome this, the authors employ a real IoT dataset related to room occupancy detection, which includes environmental features such as temperature, humidity, light, and CO_2 levels, allowing a more realistic evaluation of quantum approaches.

The study formulates the problem as a binary classification task and compares the performance of classical and quantum models. In particular, it evaluates a classical Support Vector Machine (SVM) with an RBF kernel, a standard Quantum Kernel (QK), and the more advanced Projected Quantum Kernel (PQK). The PQK approach works by encoding classical data into quantum states, performing quantum measurements, and then projecting the results back into a classical feature space, which helps reduce the complexity typically associated with high-dimensional quantum Hilbert spaces. This design aims to preserve useful quantum properties while maintaining computational feasibility.

To assess these models, the authors conduct extensive experiments using different quantum feature encoding strategies, ranging from simple rotation-based encodings to more complex circuits involving entanglement. The evaluation is performed using metrics such as accuracy, F1-score, and model complexity (measured by the number of support vectors), along with 10-fold cross-validation

to ensure robustness. The results show that PQK achieves performance comparable to, and in some cases slightly better than, the classical SVM, while consistently outperforming the standard quantum kernel. However, the study does not find clear evidence of a definitive quantum advantage over classical methods.

An important observation is that simpler, shallow quantum circuits often perform better than more complex ones, suggesting that increasing circuit depth and entanglement does not necessarily improve model performance. Additionally, PQK models tend to require fewer support vectors than classical SVMs, indicating better generalization and more compact representations. The authors also note that a small amount of measurement noise (shot noise) can sometimes improve performance, which is an interesting and somewhat counterintuitive finding. From a computational perspective, PQK reduces the quantum computational burden compared to traditional quantum kernels by shifting more processing to the classical domain, making it a more practical approach for near-term quantum devices.

Overall, the paper demonstrates that projected quantum kernels provide a promising hybrid framework for applying quantum machine learning to real-world IoT data. While the results do not establish a clear quantum advantage, they highlight the importance of encoding strategies and circuit design, and show that efficient, hybrid quantum-classical methods like PQK can achieve competitive performance with improved model efficiency.

Fatemeh Sarhaddi [1] provides a comprehensive overview of how Large Language Models (LLMs) are being integrated into Internet of Things (IoT) systems and identifies emerging research directions at the intersection of these two fields. The authors first highlight the limitations of traditional IoT data analytics approaches, which are typically based on rule-based systems or conventional machine learning models that lack contextual understanding and reasoning capabilities. In contrast, LLMs introduce powerful abilities such as natural language understanding, semantic reasoning, and knowledge integration, making them suitable for more intelligent and adaptive IoT applications.

The survey categorizes existing works based on how LLMs are utilized within IoT pipelines. These include applications where LLMs are used for data interpretation, decision-making, human–machine interaction, and automation. In many studies, raw sensor data or preprocessed outputs from

machine learning models are transformed into structured or textual representations that can be processed by LLMs, enabling higher-level reasoning and more explainable outputs. The paper also discusses the role of LLMs in generating natural language explanations from IoT data, improving system transparency and usability, especially in domains such as smart homes, healthcare monitoring, and industrial IoT.

From an architectural perspective, the survey reviews different integration strategies, including cloud-based LLM deployment, edge-based or offline LLMs, and hybrid frameworks that combine local IoT processing with remote LLM services. A key focus is on edge intelligence, where lightweight or optimized LLMs are deployed close to IoT devices in order to reduce latency, preserve privacy, and minimize dependence on internet connectivity. This is particularly important for real-time applications and smart home environments.

In terms of results and findings, the survey reports that incorporating LLMs into IoT systems can significantly enhance context awareness, decision quality, and user interaction, especially when combined with outputs from traditional ML models. However, it also emphasizes that current implementations face several challenges, including high computational cost, memory requirements, latency issues, and concerns related to data privacy and security. Additionally, the effectiveness of LLMs heavily depends on how IoT data is represented and fed into the model, with structured and well-designed prompts playing a critical role.

The paper concludes that while LLMs have strong potential to transform IoT systems from data-driven to knowledge-driven frameworks, the field is still in its early stages. It suggests future research directions such as developing lightweight LLMs for edge devices, improving multimodal data integration (sensor + text), enhancing explainability, and designing efficient pipelines that connect IoT data processing with LLM-based reasoning. Overall, the survey positions LLMs as a promising layer on top of traditional IoT analytics, enabling more intelligent, interpretable, and autonomous smart environments.

Ngoc Tuong Vy Nguyen [4] investigates the integration of Large Language Models (LLMs) into smart environment systems to enhance the interpretation of sensor data. The main objective of the study is to address the limitations of traditional IoT analytics, which typically rely on numerical outputs and

lack semantic understanding, by introducing LLMs as a reasoning and interpretation layer on top of sensor-driven systems.

The proposed approach follows a hybrid pipeline in which raw sensor data collected from smart environments (such as motion sensors, environmental sensors, and activity monitoring devices) is first processed using conventional data processing or machine learning techniques. These methods extract structured outputs, including activity labels, patterns, or statistical features. The structured information is then provided as input to an LLM, which performs higher-level reasoning and transforms the data into human-understandable insights. This enables the system to generate contextual explanations, interpret user behavior, and support intelligent decision-making in smart environments.

From a system design perspective, the paper emphasizes the importance of properly formatting sensor data into textual or structured prompts that LLMs can effectively interpret. It also discusses the potential deployment of LLMs in edge or near-device environments to reduce latency and improve responsiveness, particularly in real-time smart home scenarios. The integration allows the system to move beyond simple prediction toward semantic understanding and interaction.

The experimental findings indicate that incorporating LLMs significantly improves the interpretability and usability of smart environment systems. In particular, the LLM-based approach enables more accurate and context-aware explanations of sensor events and user activities compared to traditional methods. The results show that LLMs can successfully bridge the gap between low-level sensor signals and high-level semantic understanding, making IoT systems more interactive and user-friendly. However, the study also highlights challenges such as computational overhead, dependency on prompt design, and the need for efficient deployment strategies, especially for resource-constrained environments.

Overall, the paper demonstrates that LLMs can act as an intelligent reasoning layer on top of IoT data pipelines, enhancing explainability and contextual understanding without necessarily replacing traditional machine learning models. This hybrid approach represents a promising direction for developing more adaptive and human-centric smart environment systems.

Venkatesh [5] proposes Edge-FIT, a novel framework for deploying large language models (LLMs) in resource-constrained smart home environments while ensuring data privacy. Traditional LLM training is computationally expensive and often requires centralizing sensitive data, which is infeasible in IoT and home automation scenarios. Edge-FIT addresses this by combining federated learning with 4-bit quantized low-rank adaptation (QLoRA), enabling LLMs to be fine-tuned locally on edge devices and gateways without transmitting private data to the cloud.

The framework implements a two-tier architecture: larger models such as LLaMA-2(7B) are fine-tuned on gateway devices for more complex reasoning, while smaller models run directly on constrained devices to perform lightweight tasks. Evaluation on the Dolly 15k IoT dataset shows that Edge-FIT achieves an F1-score of approximately 0.89, comparable to centralized training, while the smaller model variant provides a practical trade-off between performance and efficiency. The study demonstrates that Edge-FIT can effectively enable privacy-preserving, edge-based intelligence in smart homes, making LLM applications feasible in IoT settings.

Key contributions include:

a) A federated instruction-tuning framework optimized for quantized LLMs in edge environments.
b) A demonstration of LLM performance close to centralized models while preserving user privacy.
c) Practical guidelines for deploying both large and lightweight LLMs across smart home devices.

Tarek Ali [6] proposes HuntGPT, a hybrid system that combines classical machine learning, Explainable AI (XAI), and large language models to improve cybersecurity monitoring. The framework uses a Random Forest classifier for anomaly detection on network traffic and integrates XAI techniques such as SHAP and LIME to provide interpretable explanations for detected threats. Additionally, an LLM (GPT-3.5 Turbo) is employed as a conversational interface, allowing analysts to query and understand anomalies in natural language.

The study evaluates HuntGPT using standard network datasets and six response quality metrics, demonstrating that the LLM-based interface enhances interpretability and user trust compared to

traditional ML systems. Results show that combining ML-based detection with XAI and LLMs produces actionable and understandable outputs, making security monitoring more accessible to human operators. Overall, HuntGPT highlights the potential of LLM-assisted, explainable anomaly detection for real-time, practical applications in cybersecurity systems.

Large Language Models (LLMs) are increasingly applied to extract information from complex textual sources, but their outputs can be prone to inaccuracies, including hallucinations or misaligned content [7]. These issues are particularly challenging when the extracted text is non-contiguous or rearranged relative to the original document, making traditional word-level verification methods insufficient for ensuring the accuracy of the extracted content. This limitation is especially critical in domains requiring high precision, such as scientific, biomedical, or technical texts.

To address this problem, Booeshaghi and Streets [7] propose a token-level alignment framework that leverages LLM-specific tokenizers and ordered token alignment. By converting both the source and extracted text into tokens consistent with the model's internal representation, the method allows for precise matching of sequences, even when content is fragmented or non-linear. This token-aware approach effectively identifies subtle discrepancies and hallucinations that word-level methods often miss, improving the reliability of extraction verification.

The study introduces benchmark datasets, BOAT for general NLP tasks and BIO-BOAT for biomedical text, to evaluate the effectiveness of their method. Experimental results demonstrate that token-level alignment increases extraction verification accuracy by approximately 50% compared to traditional word-level approaches. The framework successfully detects mismatches, omissions, and hallucinations in LLM-generated outputs, highlighting the importance of tokenization and sequence ordering in verifying extracted text.

Overall, this research provides a systematic and scalable approach to ensure the correctness of text extracted by LLMs. Its findings are particularly relevant for applications where accuracy is critical, and they offer practical guidance for improving the evaluation, reliability, and trustworthiness of LLM outputs in scientific and high-precision domains.

Proposed Methodology

1.Overview of the Proposed Method

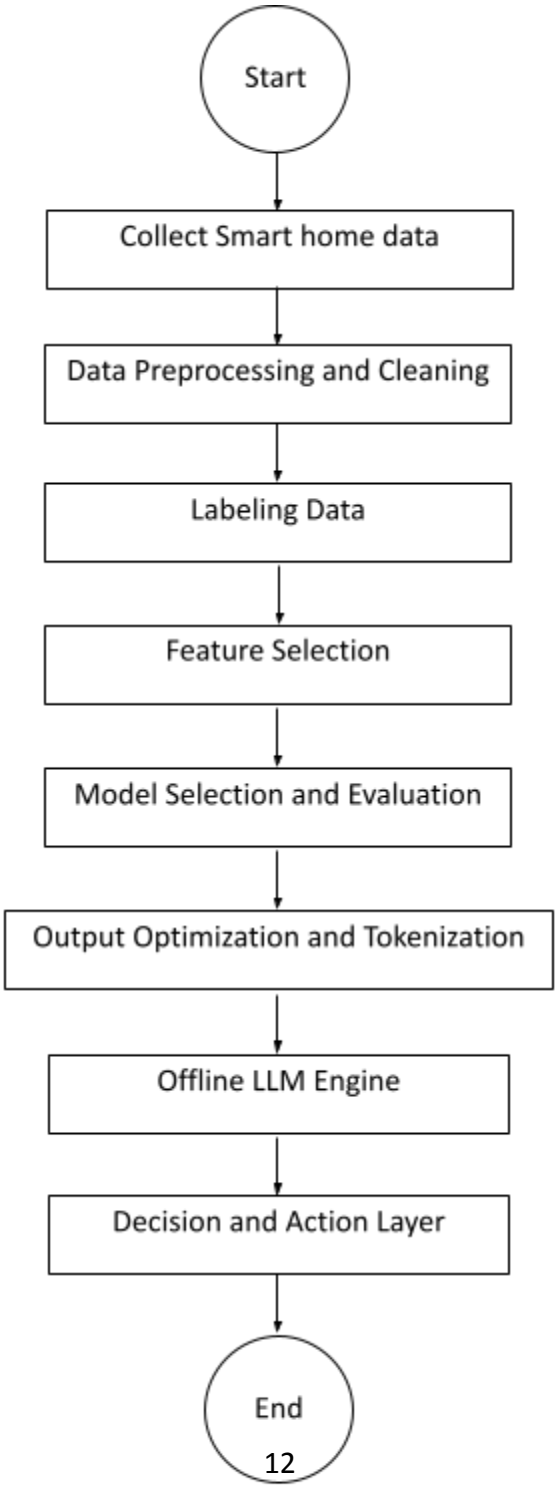
In this study, an optimized offline Large Language Model (LLM) framework is proposed for smart home environments, aiming to reduce computational cost and improve real-time responsiveness. Smart home systems generate continuous streams of data from various sensors such as temperature, energy consumption, motion detection, and appliance usage. Processing this data using traditional cloud-based LLM solutions can be computationally expensive, latency-prone, and dependent on stable internet connectivity [8] [9].

To address these limitations, the proposed methodology introduces a lightweight, offline LLM-based architecture that operates directly on local devices. The core idea is to minimize the computational burden of LLM inference by reducing input token size and optimizing prompt structures, while maintaining acceptable accuracy and responsiveness. This approach not only reduces operational costs but also enhances privacy and reliability, as sensitive data remains within the local environment [10] [11].

The system is designed to detect abnormal patterns in energy consumption, environmental conditions, and user behavior, enabling immediate feedback and automated responses. By integrating token optimization strategies with an efficient offline LLM, the framework achieves a balance between performance and resource efficiency, making it suitable for resource-constrained smart home environments [12] [13].

2.System Architecture

The proposed system consists of multiple interconnected layers designed to process, analyze, and respond to smart home data efficiently. The architecture is composed of the following components. A schematic diagram of the overall system architecture is presented first to provide a clear visual overview of the interactions between different modules. Subsequently, each component and processing stage is described in detail to explain its functionality and role within the proposed framework.



2.1 A schematic diagram of the overall system architecture

2.1 Collect Smart home data

These data were originally acquired from IoT devices operating within a real-world smart home environment and are structured as time-series observations characterized by heterogeneous data formats and non-uniform sampling frequencies.

2.2 Data Preprocessing and Cleaning

Raw sensor data is often noisy and unstructured. This layer is responsible for cleaning, filtering, and transforming the data into a structured format suitable for further processing. It also performs normalization and aggregation where necessary.

2.3 Labeling Data

Since the collected dataset does not contain predefined labels, a labeling process is required to enable further analysis and model training. In this study, an unsupervised learning approach is adopted to generate labels automatically. Specifically, clustering techniques are applied to group similar data points based on their inherent patterns and features. By identifying natural groupings within the data, each cluster is treated as a pseudo-label, allowing the transformation of unlabeled data into a labeled format. This approach enables subsequent supervised or semi-supervised learning tasks without the need for manually annotated data.

2.4 Feature Selection

Following the data labeling stage, feature selection is performed to identify the most relevant and informative features for the learning process. In smart home environments, sensor data often contains redundant and correlated features that may negatively affect model performance and increase computational cost. To address this issue, the **Minimum Redundancy Maximum Relevance (mRMR)** method is employed. This information-theoretic approach selects features that have high relevance to the generated labels while minimizing redundancy among themselves. By retaining only the most significant and non-redundant features, this step improves model efficiency, reduces dimensionality, and enhances the overall performance of subsequent learning algorithms.

2.5 Model Selection and Evaluation

After training and evaluating multiple machine learning models on the cleaned, filtered, and structured dataset, the most suitable model is selected based on its overall performance. To ensure a reliable comparison, each model is assessed using appropriate evaluation metrics such as accuracy, precision, recall, and F1-score, depending on the nature of the task. In addition to performance metrics, factors such as computational efficiency, scalability, and robustness are also considered, especially given the constraints of real-world smart home environments. The evaluation process may include techniques like cross-validation to ensure that the model generalizes well to unseen data and does not suffer from overfitting. Finally, the model that achieves the best balance between predictive performance and computational cost is chosen as the optimal candidate for deployment in the proposed system.

2.4 Output Optimization and Tokenization

Token optimization is the central contribution of this research. Once the optimal machine learning model is selected, its output is processed to ensure efficient integration with the offline LLM. In this stage, the output is refined by removing redundant and non-essential information, preserving only the most relevant and contextually important data. This optimization step

reduces computational overhead and ensures that the input provided to the LLM is concise, structured, and meaningful, which is crucial for real-time smart home applications. Following this, the optimized output is tokenized into smaller units compatible with the LLM's input format. Effective tokenization helps reduce the number of tokens and organizes the data in a way that improves processing efficiency. As a result, the offline LLM can generate responses more quickly and accurately, leading to reduced latency and improved system responsiveness while maintaining low computational cost.

Since LLM computational cost is highly dependent on the number of input tokens, reducing token count directly impacts performance and efficiency. The proposed token optimization strategy includes the following techniques:

1. **Redundancy Elimination:** Repeated or irrelevant information in the prompt is removed. For example, sensor readings that do not change significantly over time are excluded.

2. **Context Prioritization:** Only the most relevant data is included in the prompt. Priority is given to abnormal or significant changes in sensor values.

3. **Template-Based Prompting:** Instead of generating free-form text, predefined templates are used to structure the input. This reduces variability and token usage.

4. **Dynamic Token Adjustment:** The system dynamically adjusts the level of detail in the prompt based on system load and available resources.

5. **Lightweight Summarization:** Long sequences of data are summarized into shorter representations while preserving key information.

These techniques collectively reduce the number of tokens while maintaining the semantic integrity of the input, leading to faster inference and lower computational cost.

2.5 Offline LLM Engine

The optimized prompts are fed into a locally deployed LLM. The proposed system utilizes a locally deployed LLM to eliminate dependency on cloud services. The model processes the input and generates responses, predictions, or recommendations without relying on external cloud services. This configuration offers several advantages, including reduced latency, enhanced privacy, and lower operational costs.

The model is configured with the following considerations:

- Reduced maximum token limit to control resource usage
- Optimized inference settings for faster response
- Lightweight architecture suitable for edge devices
- Memory-efficient loading techniques

The offline deployment ensures that all computations are performed locally, making the system suitable for environments with limited or unreliable internet connectivity.

2.6 Decision and Action Layer

Based on the output of the LLM, this layer determines appropriate actions such as sending alerts, adjusting device settings, or notifying users about abnormal conditions.

Examples of decision logic include:

- Detecting abnormal energy consumption and notifying the user
- Identifying unusual temperature changes and triggering cooling/heating systems
- Recognizing inactivity or unexpected motion patterns

The system uses predefined rules combined with LLM-generated insights to ensure accurate and timely responses.

2.7 Cost and Performance Optimization

The proposed methodology focuses on optimizing both computational cost and system performance.

1. **Computational Cost Reduction:** By reducing the number of tokens and avoiding cloud-based processing, the system significantly lowers operational expenses.

2. **Latency Reduction:** Optimized prompts and local inference reduce response time, enabling real-time interaction.

3. **Resource Efficiency:** The system is designed to operate on devices with limited computational power, making it suitable for edge environments.

4. **Scalability:** The modular architecture allows the system to scale with additional devices and data sources without significant performance degradation.

2.8 Summary of the Proposed Methodology

In summary, this research presents a novel approach to optimizing LLM usage in smart home environments by introducing an offline architecture combined with advanced token optimization techniques. The proposed system reduces computational cost, improves response time, and enhances system reliability.

The methodology is designed to operate efficiently in resource-constrained environments while maintaining high levels of performance. By integrating data processing, token optimization, and intelligent decision-making, the framework provides a practical solution for next-generation smart home systems.

Implementation

1. Implementation Details

The system is implemented using widely adopted tools and frameworks. The development environment includes:

Development Environment	
Operating System	Windows
Development Language	Python
Tools	Jupyter Notebook
Frameworks	

Table 1.1 Development environment

2. Dataset Description

The dataset used in this study is obtained from publicly available sources on Kaggle and consists of data collected from smart home environments. These data originate from various IoT sensors deployed within residential settings, capturing real-time measurements such as temperature, humidity, energy consumption, motion detection, and device status. The dataset is primarily time-series in nature, with observations recorded at different time intervals depending on the type of sensor.

Since the data reflect real-world smart home scenarios, they may include noise, missing values, and variations in sampling frequency. Therefore, appropriate preprocessing steps are applied to clean, filter, and structure the data before further analysis. Overall, the dataset provides a

realistic representation of smart home dynamics, making it suitable for training and evaluating machine learning models within the proposed system.

Dataset Description	
Data source	Kaggle
Data type	sensor readings (temperature, humidity, etc.)
Dimensions	7
Number of samples	30001
Label Availability	Unlabeled
Supervision Type	Unsupervised

Table 2.1 Dataset Description

3. Data Preprocessing and Cleaning

This stage involves preparing the raw IoT sensor data by handling missing values, removing noise and outliers, filtering irrelevant records, and transforming the data into a clean and structured format. Additionally, normalization or standardization techniques are applied to ensure consistency across features and to improve the performance of subsequent machine learning models.

Data Preprocessing and Cleaning	
Missing values	Remove
Noise removal	Remained
Converting data	Structured format
Normalization	Done
Visualization	Show diagrams in the attachment section

Table 3.1 Data Preprocessing and Cleaning

Below is a sample of data before and after processing:

Location	Behavior	Temperature	Humidity	Noise(db)	Light intensity	Motion level
Outdoor Campus	walking	25.84	80.38	62.75993351	588.6105937	6.34
Bedroom	sleeping	27.76	66.51	26.08846275	61.35044924	1.95
Cafeteria	eating	31.06	50.28	48.70018676	230.2772932	3.47

Table 3.2 Sample of data

temperature	Light intensity	Motion level	Location Bedroom	Location Crowded place
25.84	62.75993	588.62	6.34	False
27.76	26.08846	61.36	1.95	False
31.06	48.70019	230.28	3.47	False

Location Cafe	Location Cafeteria	Location Dining Room	Location Library	Location Meeting Room
False	False	False	False	False
True	False	False	False	False
False	False	False	False	False

Noise (db)	Location Outdoor Campus	Location Park	Location Street	Location Study Room	Location Traffic
80.38	False	False	False	False	False
66.51	False	False	False	False	False

50.28	False	False	False	False	False
Location Office	**Behavior eating**	**Behavior sleeping**	**Behavior stressed**	**Behavior studying**	**Behavior walking**
False	False	False	False	False	False
False	False	False	False	True	False
False	False	False	True	False	False

4. Labeling Data

In this study, an unlabeled dataset was analyzed to uncover its underlying structure without prior knowledge of class labels. Since no ground truth was available, unsupervised learning techniques were employed, with a particular focus on clustering methods. Several clustering approaches were explored and evaluated based on their ability to produce meaningful and interpretable groupings. Among these, Hierarchical Clustering proved to be the most effective, as it provided a clear representation of the relationships between data points through a tree-like structure (dendrogram), enabling a deeper understanding of the data distribution.

The hierarchical approach was especially suitable because it does not require pre-specifying the number of clusters, unlike methods such as k-means. This flexibility allowed for a more natural exploration of the dataset's structure, revealing nested groupings and similarities at multiple levels of granularity. By analyzing the resulting dendrogram, an appropriate number of clusters was determined, and the data was partitioned accordingly. The clusters formed using this method demonstrated strong internal cohesion and clear separation, indicating that the

Table 3.3 Sample of data after cleaning, filtering, and transforming the data into a structured format

Overall, the application of Hierarchical Clustering provided valuable insights into the organization of the unlabeled data and established a solid foundation for further analysis. The results suggest that this method is a robust choice for exploratory data analysis in scenarios where labeled data is unavailable. Future work may involve validating these clusters using domain knowledge or applying semi-supervised techniques to refine the model and improve its applicability in real-world tasks.

5. Feature Selection

Feature selection is a crucial step in data preprocessing, particularly in smart home environments where sensor data is often high-dimensional, redundant, and noisy. Selecting the most informative features not only improves model performance but also reduces computational complexity, which is especially important for resource-constrained environments such as IoT-based systems.

In this study, an information-theoretic approach is adopted for feature selection, specifically the **Minimum Redundancy Maximum Relevance (mRMR)** method. This method aims to select a subset of features that are highly relevant to the target variable while minimizing redundancy among the selected features.

The mRMR algorithm operates based on mutual information, evaluating the statistical dependency between variables. It maximizes the relevance of selected features with respect to the target labels while simultaneously minimizing the redundancy between features. This balance ensures that the selected feature subset contains the most discriminative and non-overlapping information.

Formally, the objective of mRMR can be expressed as maximizing the difference between relevance and redundancy:

$$max\left(\frac{1}{|S|}\sum_{x_i \in S} I(x_i; y) - \frac{1}{|S|^2}\sum_{x_i, x_j \in S} I(x_i, x_j)\right)$$

Formula 4.1 The mRMR objective function

where Srepresents the selected feature subset, $I(x_i; y)$denotes the mutual information between feature $x_i$and the target variable y, and $I(x_i; x_j)$represents the mutual information between pairs of features.

The choice of mRMR is motivated by its effectiveness in handling high-dimensional IoT data, where many features may be correlated due to the nature of sensor measurements. By reducing redundancy and preserving the most relevant information, mRMR enhances the efficiency and accuracy of subsequent learning models.

Furthermore, in this study, since the original dataset lacks labeled data, pseudo-labels generated through clustering techniques are utilized as the target variable for the mRMR feature selection process. This enables the integration of unsupervised and supervised learning paradigms, improving the overall quality of the selected features.

6. Model Selection and Evaluation

After this section, supervised learning models were applied to a labeled version of the dataset in order to evaluate classification performance. According to related approaches in the literature, these models were selected as they are suitable for this type of problem. These three supervised learning models were evaluated to determine the most suitable approach for classification. The selected models include:

- Decision Tree (DT)
- Random Forest (RF)
- Extreme Gradient Boosting (XGBoost)

These models were chosen due to their effectiveness in handling structured tabular data and their ability to model complex relationships in IoT datasets.

The dataset was divided into training and testing subsets to ensure unbiased evaluation. In addition, cross-validation was applied to improve the robustness of the results and reduce overfitting.

To ensure a fair comparison, all models were evaluated using the same performance metrics:

- Accuracy
- Precision
- Recall
- F1-score

These metrics provide a comprehensive evaluation of classification performance, particularly in scenarios where class distributions may vary.

The performance results of the three models are summarized in Table 6.1.

Model	Accuracy	Precision	Recall	F1-Score
Decision Tree	1.00	0.99	0.98	
Random Forest	1.00	0.99	0.98	
XGBoost	1.00	0.99	0.99	

Table 6.1 The performance results of the three models

The results indicate that all three models achieve very high performance, with accuracy values close to 1.00. This suggests that the dataset is highly structured and that the classification task is relatively straightforward.

No significant performance gap is observed between Decision Tree, Random Forest, and XGBoost, indicating that all models generalize well on the given data. Therefore, it is not possible to conclude that the Decision Tree model suffers from overfitting based on these results alone.

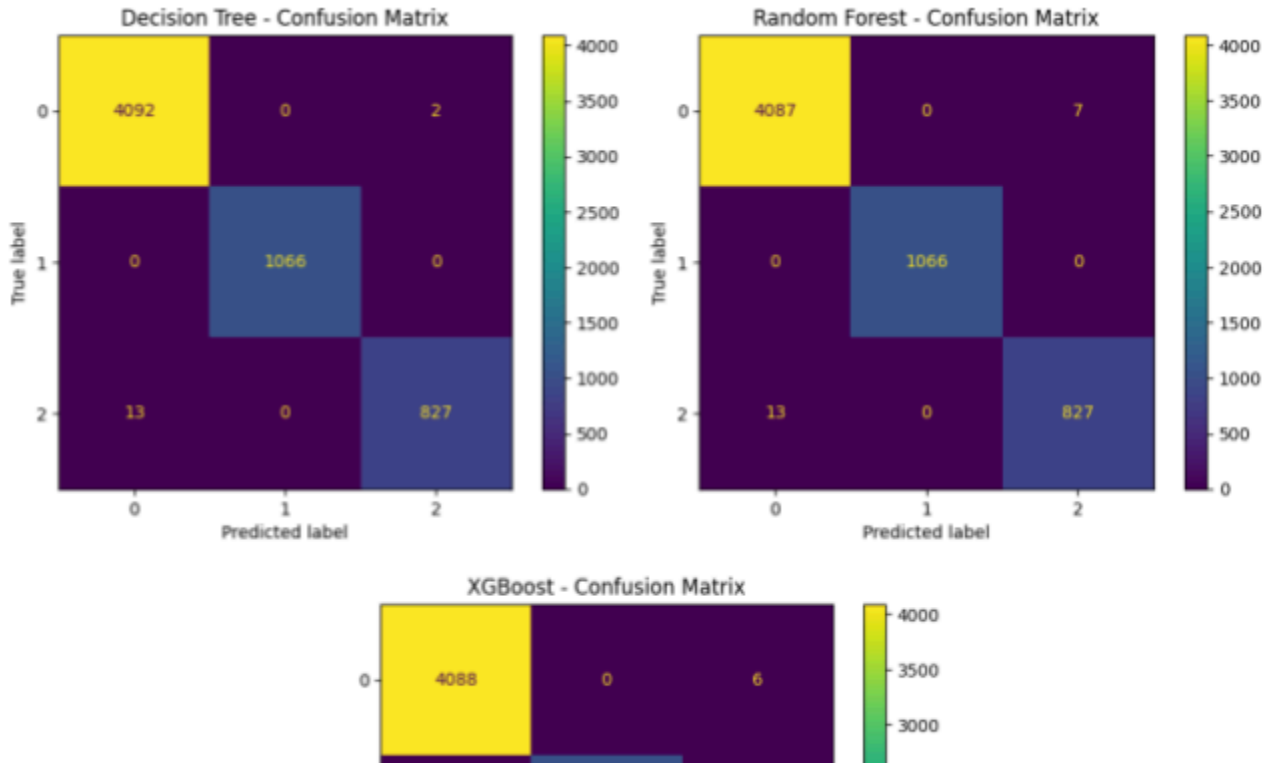
However, ensemble methods such as Random Forest and XGBoost are generally preferred in real-world applications due to their higher robustness and stability when dealing with more complex or noisy datasets.

Although all models show similar performance, XGBoost slightly outperforms the others in recall and provides a more robust boosting mechanism. In addition, XGBoost is known for its ability to handle complex patterns and reduce overfitting through regularization techniques. Therefore, it was selected as the final model.

To further analyze the performance of XGBoost, additional evaluation plots were generated:

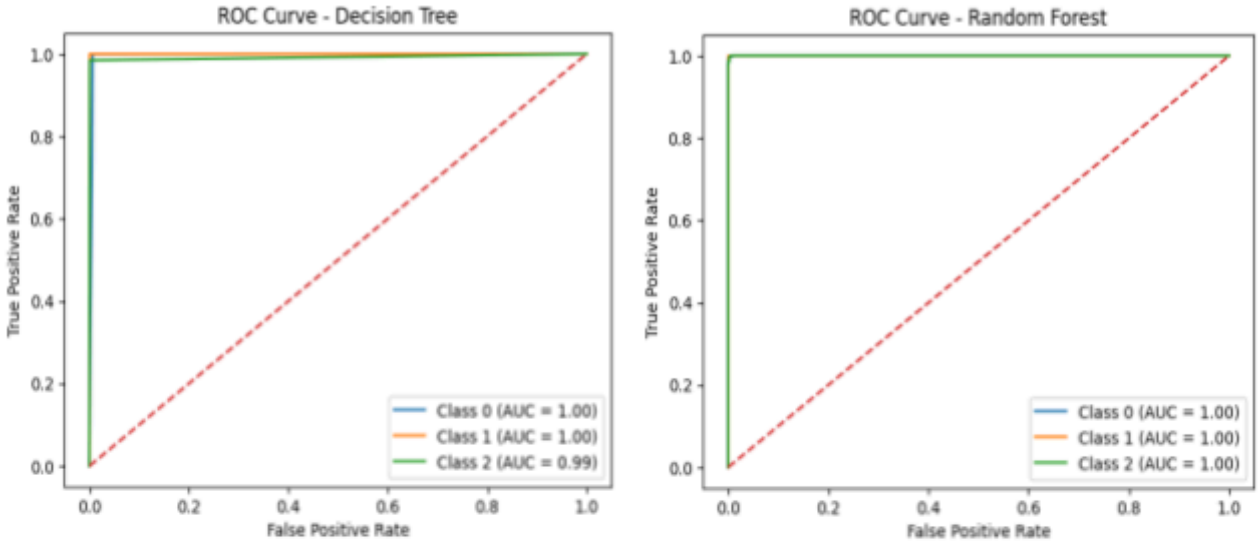
- **Confusion Matrix (Figure 6.1):**

  Shows the classification accuracy across different classes and highlights the model's ability to correctly classify most instances.



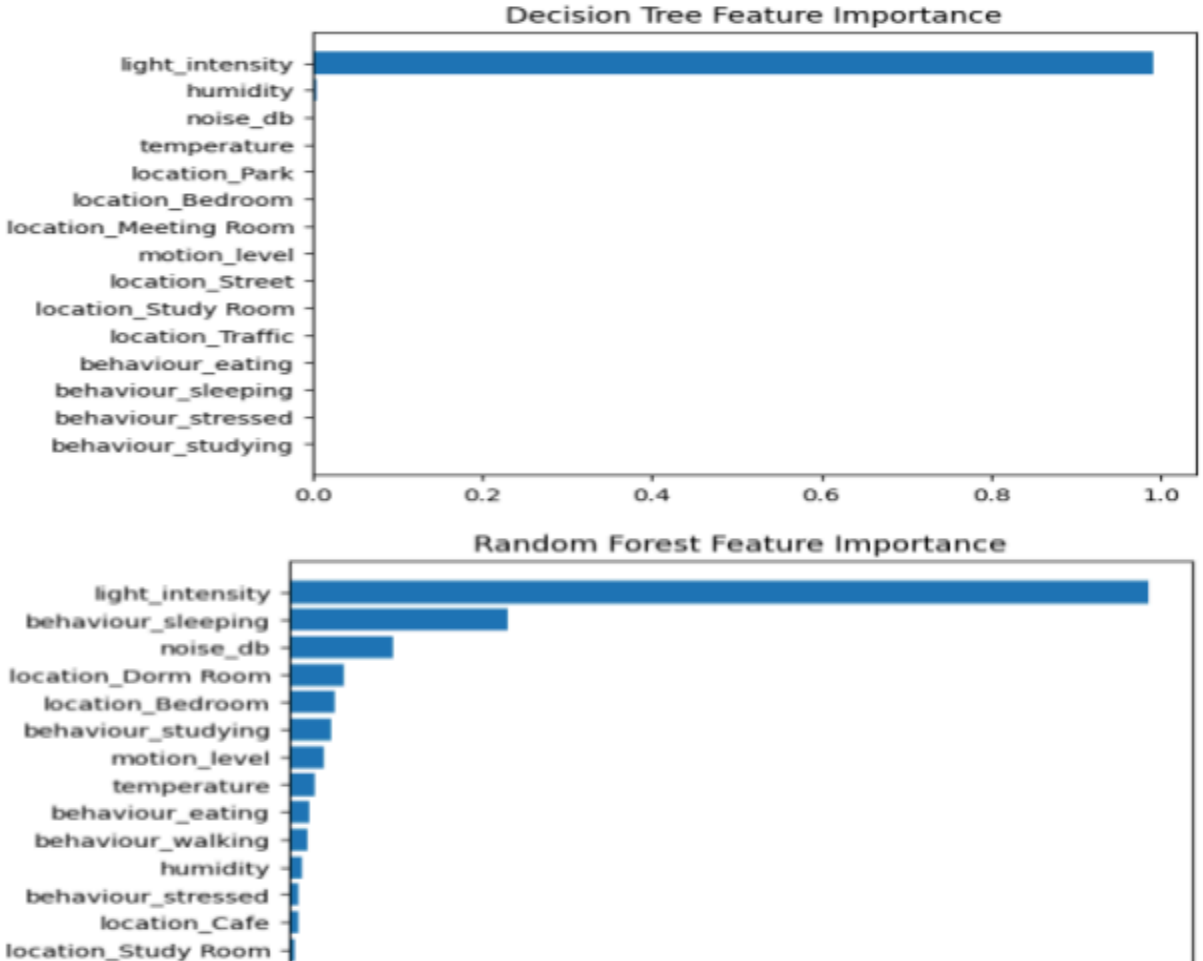
- **ROC Curve (Figure 6.2):**

  Demonstrates a strong trade-off between true positive rate and false positive rate, indicating high discriminative ability.



- **Feature Importance (Figure 6.3):**

  Identifies the most influential features contributing to model predictions, confirming the effectiveness of the feature selection process.



7. Output Optimization and Tokenization

The raw output generated by the classification model contains redundant structural metadata and detailed numerical representations (e.g., arrays) that are not necessary for downstream reasoning by the offline LLM. Therefore, an output optimization process was applied to transform the raw output into a compact, structured, and token-efficient format.

```
output:{
'schema_version': '1.0',
'task_id': 'iot_event_classification',
'timestamp': '2026-05-16T09:42:25.367462',
'prediction': {
        'label': 0,
         'confidence': 1.0
            },
'signal_summary': {
        'top_features': [
                        {'name': 'temperature',
                        'impact': array([0, 1, 2],
                        dtype=int64)
```

Step 1: Remove unnecessary metadata

Remove:

- schema_version
- task_id
- timestamp

Step 2: Simplify prediction

From:

```
"prediction": {"label": 0, "confidence": 1.0}
```

To:

```
label: 0, confidence: 1.0
```

Step 3: Extract meaningful features

From:

```
'impact': array([0,1,2])
```

To:

```
top_features: temperature, humidity, noise
```

LLM does NOT need arrays. Instead of them, Keep only feature names or derive importance.

Step 4: Convert to compact format

Final structured output:

```
label:0 | conf:1.0 | features: temperature, humidity, noise
```

Here is Optimization Codes:

```
def optimize_output(data):

    # Extract prediction
    label = data['prediction']['label']
    confidence = data['prediction']['confidence']

    # Extract top features (ignore arrays)
    features = [
    f['name'] for f in data['signal_summary']['top_features']
    ]

    # Create compact output
    optimized = f"label:{label} | conf:{confidence} | features:{','.join(features)}"

    return optimized
```

The proposed optimization approach aligns with recent research in LLM efficiency, which emphasizes reducing token redundancy, using structured representations, and filtering irrelevant information. By converting verbose outputs into compact symbolic formats, the number of tokens is significantly reduced, leading to improved inference speed and lower computational cost. This is particularly important in offline and edge-based environments, where resource constraints require efficient processing.

Here is Tokenization Step (Offline LLM):

```
from transformers import AutoTokenizer


tokenizer = AutoTokenizer.from_pretrained("your-local-llm")
optimized_text = optimize_output(optimize_output_result)


tokens = tokenizer(optimized_text)
print(tokens)
```

The proposed optimization approach aligns with recent research in LLM efficiency, which emphasizes reducing token redundancy, using structured representations, and filtering irrelevant information. By converting verbose outputs into compact symbolic formats, the number of tokens is significantly reduced, leading to improved inference speed and lower

computational cost. This is particularly important in offline and edge-based environments, where resource constraints require efficient processing.

This optimization enables the offline LLM to focus only on the most relevant information, improving both reasoning efficiency and real-time responsiveness in smart home environments.

8. Offline LLM Engine

9. Decision and Action Layer

Evaluates

Concludes

References

[1] N. T. N. A. Z. P. H. S. T. H. F. P. N. Fatemeh Sarhaddi, "LLMs and IoT: A Comprehensive Survey on Large Language Models and the Internet of Things," *TechRxiv (preprint), 2025,* p. 29, 2026.

[2] A. M. Y. B. Mingyu Zong, "Integrating large language models with internet of things: applications," *Discover Internet of Things (Springer journal focused on IoT research),* vol. 5, no. 1, pp. 1-15, 2025.

[3] L. M. C. M. F. P. L. S. J. S. A. V. Francesco D'Amore, "Assessing Projected Quantum Kernels for the Classification of IoT Data," *Array,* vol. 29, 2025.

[4] Y. Y. Ngoc Tuong Vy Nguyen, "Large Language Models for Sensor Data Interpretation in Smart Environments," in *IEEE Conference Publication*, 2025.

[5] V. R. K. L. K. N. K. Vinay Venkatesh, "Edge-FIT: Federated Instruction Tuning of Quantized LLMs for Privacy-Preserving Smart Home Environments," *arXiv Preprint,* p. 2025.

[6] P. K. Tarek Ali, "HuntGPT: Integrating Machine Learning-Based Anomaly Detection and Explainable AI with Large Language Models (LLMs)," *arXiv Preprint,* 2023.

[7] A. S. Sina Booeshaghi, "Verifying LLM-extracted text with token alignment," *bioRxiv (preprint server),* 2025.

[8] Z. H. K. Erçin Dinçer, "Real-Time and Offline Large Language Models on Edge Devices," *Preprints.org,* 2025.

[9] Z. G. L. Z. ,. S. Y. ,. Z. G. Rui Wang, "Empowering large language models to edge intelligence: A survey of edge efficient LLMs and techniques," *Computer Science Review,* vol. 57, 2025.

[10] T. A. V. O. B. K. O. A. O. A. O. K. Serhiy O. Semerikov, "“Edge intelligence unleashed: a survey on deploying large language models in resource-constrained environments," *Journal of Edge Computing,* vol. 4, no. 2, pp. 179-233, 2025.

[11] Q. L. T. Y. P. K. Z. L. Z. P. Minhang Liang, "LLM-based Exploitation of Edge Data in Modern Power Systems," *Journal of Modern Power Systems and Clean Energy,* vol. 14, no. 1, 2026.

[12] F. F. Gabriele De Vito, "The role of Large Language Models in addressing IoT challenges: A systematic literature review," *Future Generation Computer Systems (Elsevier) — often abbreviated FGCS,* vol. 171, 2025.

[13] A. A. Yazan Otoum, "LLMs meet Federated Learning for Scalable and Secure IoT Management," in *IEEE Global Communications Conference (GLOBECOM) 2025*, 2025.

[14] Y. Z. Y. J. T. T. W. Shuyue Wei, "Federated reasoning LLMs: a survey," *Frontiers of Computer Science,* vol. 19, no. 12, 2025.

[15] Ł. J. Nur Keleşoğlu, "When LLMs speak ZigBee: exploring low-latency and reasoning models for network traffic generation," *Scientific Reports,* vol. 16, 2026.

[16] R. S. R. Lavanya Elluri, "Exploring IoT privacy compliance research: a topic modeling approach," *International Journal of Information Security,* vol. 25, 2026.

Attached